Practical No. 6 : Backtracking Search — 8-Queens Problem

```
def is_safe(board, row, col, n):
    # Check this column on upper rows
    for i in range(row):
        if board[i] == col:
            return False
        # Check diagonals
        if abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve_n_queens(board, row, n, solutions):
    if row == n:
        solutions.append(board.copy())
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            solve_n_queens(board, row + 1, n, solutions)
            board[row] = -1

def print_board(board, n):
    for r in range(n):
        line = ""
        for c in range(n):
            line += "Q " if board[r] == c else ". "
        print(line)
    print()
```

```
def eight_queens():
    n = 8
    board = [-1] * n
    solutions = []
    solve_n_queens(board, 0, n, solutions)
    print(f"Total solutions found for {n}-Queens: {len(solutions)}\n")
    print("First solution:\n")
    print_board(solutions[0], n)

if __name__ == "__main__":
    eight_queens()
```